

RAG (Retrieval Augmented Generation) Cheat Sheet

14 tables · 126 concepts · Updated 2026-08-01 · [cheatgrid.com](#)

Retrieval-Augmented Generation (RAG) is an LLM application pattern that **retrieves external knowledge** at query time and injects it into the model's context to produce grounded answers. Practitioners use RAG to reduce hallucinations, incorporate **fresh/private data**, and make outputs auditable via sources/citations. A useful mental model is that RAG is two coupled systems: an **information retrieval** system (indexes, rankers, filters) and a **response synthesis** system (prompting, citations, formatting). By 2026, production RAG has evolved well beyond naive chunk-and-retrieve: the dominant pattern is **agentic RAG**, where the LLM itself decides when, what, and how to retrieve. Most "RAG problems" are retrieval problems first: if the right evidence never makes it into context, generation quality cannot recover it.

CONTENTS

RAG Building Blocks (Conceptual)	1
Chunking and Splitting	2
Embedding Models	3
Dense Indexing and ANN	4
Vector Databases	4
Sparse and Hybrid Retrieval	5
Query Transformation	6
Reranking and Fusion	6
Query Engines and Answer Grounding	7
Advanced RAG Architectures	8
Storage, Persistence, and Caching	8
RAG Evaluation Metrics and Frameworks	9
Observability and Security	10
Multimodal RAG	10
References	11

TABLE 1

RAG Building Blocks (Conceptual)

Every RAG pipeline is assembled from the same handful of stages, run in order from ingesting raw data all the way to evaluating the final answer. Knowing this skeleton makes the rest of the cheat sheet easier to navigate — each later table simply zooms into one of these stages and lists the concrete tools you can plug in there.

Stage	Example	Description
Retrieval-Augmented Generation (RAG)	<code>answer = LLM(question, context=top_k_docs)</code>	Generates with retrieved evidence rather than relying only on parametric memory.
Ingestion	<code>docs = loader.load_data()</code>	Reads source data and converts it into document objects for downstream processing.
Chunking	<code>splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)</code>	Splits documents into smaller units so retrieval can target the right passages.
Embedding	<code>emb = client.embeddings.create(model="text-embedding-3-small", input="...")</code>	Maps text to vectors for similarity search in dense retrieval.

Stage	Example	Description
Indexing	<code>index.add(xb)</code>	Builds a search structure over vectors (exact or ANN) to enable fast retrieval.
Retrieval	<code>docs = retriever.invoke("...")</code>	Selects candidate chunks/documents relevant to the query.
Reranking	<code>reranked = co.rerank(query=q, documents=texts)</code>	Reorders retrieved candidates to improve top-k quality .
Synthesis	<code>resp = query_engine.query("...")</code>	Produces the final answer from retrieved context (often via an LLM).
Citations	<code>resp = citation_engine.query("...")</code>	Attaches sources to claims (usually at chunk-level granularity).
Evaluation	<code>score = faithfulness</code>	Measures retrieval + generation quality with task-appropriate metrics.

TABLE 2

Chunking and Splitting

How you slice documents into chunks quietly decides how good your retrieval can ever be: too big and you bury the answer in noise, too small and you lose the surrounding context. These splitters range from the dependable recursive-character default to semantic and structure-aware options, plus the parent/child, contextual, and late-chunking tricks that fix the boundary problems naive splitting creates.

Splitter	Example	Description
RecursiveCharacterTextSplitter	<code>RecursiveCharacterTextSplitter(separators=["\n\n", "\n", " ", ""], chunk_size=512, chunk_overlap=50)</code>	• Default general-purpose splitter that tries separators in order to form sized chunks • a 2026 benchmark across 50 academic papers found it the strongest single default, ahead of fixed-size and semantic chunking.
SemanticChunker	<code>SemanticChunker(embeddings, breakpoint_threshold_type="percentile")</code>	• Splits at semantic boundary breaks in embedding space • wins on retrieval recall but needs a minimum chunk-size floor, since unfloored fragments can be too small to give the LLM enough context to answer correctly.
TokenTextSplitter	<code>TokenTextSplitter(chunk_size=512, chunk_overlap=64)</code>	Splits by token count (useful when you need predictable model-context usage).
SentenceSplitter	<code>SentenceSplitter(chunk_size=512, chunk_overlap=50)</code>	Splits text into sentence-based chunks with size/overlap controls.
MarkdownHeaderTextSplitter	<code>MarkdownHeaderTextSplitter(headers_to_split_on=[("#", "h1"), ("#", "#", "h2")])</code>	Splits Markdown while preserving header structure as metadata.
HTMLHeaderTextSplitter	<code>HTMLHeaderTextSplitter(headers_to_split_on=("h1", "h1"), ("h2", "h2"))</code>	Splits HTML by header tags to keep section semantics.
CharacterTextSplitter	<code>CharacterTextSplitter(separator="\n\n", chunk_size=512, chunk_overlap=50)</code>	Simple splitter using a fixed separator and target chunk size.
Contextual Chunking	<code>contextualized_chunk = f"CONTEXT: {llm(doc, chunk)}\n\n{chunk}"</code>	• Prepends an LLM-generated description of where each chunk fits in its document • Anthropic found this reduces retrieval failures by 49%.

Splitter	Example	Description
Late Chunking	<pre>chunker = LateChunker(embedding_model=long_context_model, chunk_size=512)</pre>	- Embeds the whole document first with a long-context embedding model, then splits the resulting token embeddings into chunks - each chunk's embedding retains surrounding pronouns, headers, and cross-references that early chunking strips away; needs the full document to fit the embedding model's context window.
ParentDocumentRetriever	<pre>ParentDocumentRetriever(vectors_tore=vs, docstore=store, child_splitter=small_splitter)</pre>	Indexes small child chunks for precise recall but returns the larger parent chunk to the LLM for richer context.
chunk_overlap	<pre>RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)</pre>	Repeats tail tokens/characters across chunks (typically 10-20% of chunk size) to reduce boundary misses.
Metadata	<pre>Document(page_content=text, metadata={"source": url, "page": 12})</pre>	Carries provenance and filters (e.g., doc id, page, section) through retrieval.

TABLE 3

Embedding Models

The embedding model is what turns text into the vectors your search runs on, so its quality sets the ceiling on retrieval accuracy. The choice usually comes down to a few trade-offs: proprietary APIs like OpenAI, Cohere, Voyage, and Gemini versus self-hostable open models like BGE-M3, dimension count and cost, and how well each handles non-English or domain-specific corpora.

Model	Example	Description
text-embedding-3-small	<pre>client.embeddings.create(model="text-embedding-3-small", input="...")</pre>	- OpenAI's cost-effective embedding model 1536-dim, supports dimension reduction to 512 with minimal recall loss.
text-embedding-3-large	<pre>client.embeddings.create(model="text-embedding-3-large", input="...")</pre>	- OpenAI's high-accuracy model 3072-dim, supports reduction to 256+ - strong English performance across MTEB.
Cohere embed-v4	<pre>co.embed(texts=[...], model="embed-v4.0", input_type="search_document")</pre>	- Multimodal and multilingual 1536-dim, scores ~65.2 on MTEB - best for non-English corpora and mixed-modality inputs.
Voyage voyage-4-large	<pre>vo.embed(texts, model="voyage-4-large", input_type="document", output_dtype="int8")</pre>	- Voyage AI's current flagship embedding model - natively produces Matryoshka embeddings and supports int8/binary output via <code>output_dtype</code> for cheaper storage - domain-specific variants (code, finance, law) are also offered separately.
Gemini Embedding 2	<pre>client.models.embed_content(model="gemini-embedding-2", contents="...")</pre>	- Google's first multimodal embedding model: text, image, video, audio, and PDF share one vector space 3072-dim by default (Matryoshka-trained; auto-normalizes truncated 768/1536-dim outputs) - released April 2026, ranking at the top of the MTEB English leaderboard.
BGE-M3	<pre>model = BGEM3FlagModel("BAAI/bge-m3"); model.encode(texts)</pre>	- Open-source - supports dense, sparse, and multi-vector retrieval in one model - self-hostable on a single GPU.
E5-Mistral	<pre>model = SentenceTransformer("intfloat/e5-mistral-7b-instruct")</pre>	- Open-source 7B model - competitive with proprietary models on MTEB instruction-tuned for strong passage retrieval.

TABLE 4

Dense Indexing and ANN

Once you have millions of vectors, scanning all of them per query is too slow: approximate nearest-neighbor (ANN) indexes trade a sliver of recall for orders-of-magnitude faster search. Here you'll find the graph- and partition-based structures (HNSW, IVFFlat) that power production vector DBs, the exact FAISS baselines, the quantization and dimension-reduction tricks that shrink memory, and the distance metrics that define "similar."

Index	Example	Description
HNSW	<code>CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops);</code>	• Graph-based ANN structure • the de-facto standard in production vector DBs due to low query latency at high recall.
IVFFlat	<code>CREATE INDEX ON items USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);</code>	• Inverted-file ANN: partitions vectors into lists and probes a subset at query time • lower build cost than HNSW.
IndexFlatL2	<code>index = faiss.IndexFlatL2(d)</code>	• Exact L2 distance brute-force baseline in FAISS • useful for small corpora or ground-truth evaluation.
IndexFlatIP	<code>index = faiss.IndexFlatIP(d)</code>	Exact inner-product search baseline in FAISS.
Product Quantization (PQ)	<code>index = faiss.IndexPQ(d, m, nbits)</code>	Compresses vectors into short codes (6-10x compression) to reduce memory and speed up search.
Scalar Quantization (SQ)	<code>ScalarQuantization(type=ScalarT ype.INT8)</code>	• Quantizes each dimension to 8-bit integers, a 4x storage cut • lighter than PQ with minimal recall loss • widely used in Qdrant.
Binary Quantization	<code>output_dtype="binary" (each dimension packed to 1 bit)</code>	• Compresses each dimension to a single bit, a 32x storage cut versus 32-bit floats • a fast Hamming-distance prefilter typically reranks top candidates against the original float vectors to recover most of the lost recall.
Matryoshka Representation Learning (MRL)	<code>resized = embd_float[:, :256] (truncate a 1024-dim vector)</code>	• Trains embeddings so a leading prefix of dimensions is itself a valid, slightly-lower-quality embedding • lets you shrink vectors for storage and speed without re-invoking the embedding model; used by OpenAI, Voyage, and Gemini embedding models.
Cosine	<code>ORDER BY embedding <=> \$1 LIMIT 10</code>	Common similarity for normalized embeddings (implemented as distance operator in pgvector).
DotProduct	<code>scores = x @ q</code>	• Inner product similarity • equivalent to cosine when vectors are L2-normalized.
Normalization	<code>faiss.normalize_L2(x)</code>	Makes cosine similarity retrieval equivalent to dot-product search.
ColBERT / Multi-Vector	<code>rag = RAGPretrainedModel.from_p retrained("colbert-ir/colbertv2 .0"); rag.search(q, k=5)</code>	• Stores per-token embeddings • scores with MaxSim (max cosine per query token) • near cross-encoder accuracy at near bi-encoder speed.

TABLE 5

Vector Databases

A vector database is where your embeddings actually live and get queried in production, bundling the ANN index with storage, metadata filtering, and operations. The real decision is which one fits your situation: pgvector if you already run Postgres, managed Pinecone for zero-ops scale, Qdrant or Weaviate for open-source flexibility, Chroma for quick prototyping.

Database	Example	Description
pgvector	<code>CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops);</code>	• PostgreSQL extension • keeps vectors and app data in the same table/transaction • the default choice if you already run Postgres, and scales well past 5M vectors when paired with the pgvector extension.
Pinecone	<code>index.upsert(vectors=[(id, emb, meta)]); index.query(vector=q, top_k=10)</code>	• Fully-managed SaaS • zero-ops serverless, auto-scales to billions of vectors • supports sparse-dense hybrid search.
Qdrant	<code>client.search("docs", query_vector=q, query_filter=Filter(...), limit=10)</code>	• Open-source Rust-native DB • rich payload filtering, scalar/binary quantization, self-hosted or Qdrant Cloud.
Weaviate	<code>collection.query.near_text("...", limit=10)</code>	• Open-source • built-in vectorization modules (auto-embeds raw text), built-in hybrid BM25+vector search • GraphQL API.
Milvus / Zilliz	<code>client.search(collection_name="docs", data=[q_emb], limit=10)</code>	• Open-source • GPU-accelerated, billion-scale distributed clusters • Zilliz Cloud is the managed version.
Chroma	<code>collection.query(query_texts=["..."], n_results=10)</code>	• Embedded or client-server • zero-setup developer experience • best for prototyping and local development.
LanceDB	<code>table.search(q_emb).limit(10).to_arrow()</code>	• Zero-copy columnar storage (Lance format) • embedded/in-process, disk-based indexing for larger-than-RAM datasets.

TABLE 6

Sparse and Hybrid Retrieval

Dense vectors are great at meaning but weak at exact terms — error codes, product names, and rare keywords often slip through. Sparse methods like BM25 catch exactly those, and hybrid retrieval fuses both signals so you don't have to choose; the rest of the rows cover the filtering, fusion, and diversity controls that shape the final candidate set.

Retriever	Example	Description
HybridSearch	<code>collection.query.hybrid(query="...", alpha=0.5, limit=10)</code>	• Combines sparse and dense vector signals into one ranked list • covers both semantic similarity and exact-match needs.
BM25	$\text{score}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot (f(t,d) \cdot (k_1 + 1)) / (f(t,d) + k_1 \cdot (1 - b + b \cdot (d / (\text{avgdl})))$	• Term-based ranking model used for keyword (sparse) retrieval • strong on exact terms, error codes, and product names.
MMR	$\text{argmax}_{d \in R \setminus S} \lambda \text{sim}(d,q) - (1-\lambda) \max_{d' \in S} \text{sim}(d,d')$	• Diversifies selected chunks by trading off relevance vs redundancy • avoids returning near-duplicate passages.
MetadataFiltering	<code>where={"path":["source"],"operator":"Equal","valueString":"handbook"}</code>	Restricts candidates to documents matching structured metadata predicates before vector search.
Fusion (RRF)	$\text{RRF}(d) = \sum_i 1 / (k + \text{rank}_i(d))$	• Rank aggregation to merge results from multiple retrievers without needing calibrated scores • $k=60$ is common.
TopK	<code>docs = retriever.invoke(q, config={"configurable": {"k": 10}})</code>	Controls how many candidates are returned from retrieval.
ScoreThreshold	<code>as_retriever(search_type="similarity_score_threshold", search_kwargs={"score_threshold": 0.8})</code>	Filters results by minimum similarity score before passing context to the LLM.
Similarity (BM25 config)	<code>"similarity": {"default": {"type": "BM25", "k1": 1.2, "b": 0.75}}</code>	Configures per-field BM25 scoring parameters in Elasticsearch.

TABLE 7

Query Transformation

Users rarely phrase questions the way your documents are written, and a single raw query often misses relevant chunks. These techniques reshape the query before it ever hits the index — rewriting it, expanding it into several variants, generating a hypothetical answer to search with, or breaking a complex question into sub-questions — to close that gap between how people ask and how content is stored.

Technique	Example	Description
QueryRewrite	<code>"Rewrite the question for search: ..."</code>	Converts user input into a search-optimized query string to improve vector recall.
MultiQueryRetriever	<code>mqr = MultiQueryRetriever.from_llm(retriever, llm)</code>	Uses an LLM to generate multiple query variants and unions retrieved docs for better recall.
HyDE	<code>hyp = llm("Write a passage answering: ...") docs = retriever.invoke(hyp)</code>	- Retrieves using embeddings of a hypothetical document generated from the query - helps with vague or sparse questions.
SelfQueryRetriever	<code>sq = SelfQueryRetriever.from_llm(llm, vectorstore, document_content_description, metadata_field_info)</code>	Uses an LLM to translate natural language into a structured query + metadata filters .
Decomposition	<code>subqs = ["...", "..."]</code>	Splits a complex question into sub-questions that are answered individually and combined.
Step-Back Prompting	<code>broader = llm(f"What is the more general question behind: '{q}'") docs = retriever.invoke(broader)</code>	- First retrieves for a broader abstraction of the question before the specific query - improves recall on specific questions.
Routing	<code>route = router.invoke({"question": q})</code>	- Selects a retriever/index/tool based on query intent or domain - avoids wasting retrieval on the wrong corpus.
Expansion	<code>q' = q + synonyms(q)</code>	Adds related terms/phrases to improve recall in sparse retrieval.
Rephrase	<code>docs = retriever.invoke(rephrased_q)</code>	Normalizes user phrasing to reduce retrieval mismatch from casual or ambiguous language.

TABLE 8

Reranking and Fusion

First-pass retrieval is tuned for speed and recall, so it usually pulls back more than you need with the best chunk not always on top. A reranker does a second, more expensive pass that scores each query-document pair jointly and reshuffles for precision. Anthropic measured reranking cutting retrieval failures by 67% when combined with contextual retrieval. The fusion and compression rows round out the toolkit for merging multiple ranked lists and trimming context noise.

Ranker	Example	Description
Cross-encoder reranking	<code>scores = cross_encoder.predict([(q, d) for d in docs])</code>	- Scores each query-document pair jointly with full attention for higher-precision top-k - compute cost is small vs LLM call.
Cohere Rerank	<code>co.rerank(model="rerank-v4.0", query=q, documents=texts, top_n=10)</code>	API reranker producing a relevance-ordered list with scores - Rerank 4 (Dec 2025) is a single multilingual model (100+ languages) with a 4x larger context window than Rerank 3.5, replacing the older English/multilingual-split Rerank 3.0 models - Anthropic found reranking cuts retrieval failures by 67% combined with contextual retrieval.
BGE Reranker	<code>reranker = FlagReranker("BAAI/bge-reranker-v2-m3"); scores = reranker.compute_score([(q,d) for d in docs])</code>	- Open-source cross-encoder reranker multilingual, competitive with commercial rerankers - self-hostable.

Ranker	Example	Description
Reciprocal Rank Fusion (RRF)	$RRF(d) = \sum_i \frac{1}{(k + rank_i(d))}$	- Fuses multiple ranked lists without needing calibrated similarity scores $k=60$ is a common default.
EnsembleRetriever	<pre>ens = EnsembleRetriever(retrievers=[r1, r2], weights=[0.5, 0.5])</pre>	Combines multiple retrievers and applies rank fusion to merge results.
ContextualCompressionRetriever	<pre>cc = ContextualCompressionRetriever(base_retriever=r, base_compressor=compressor)</pre>	Retrieves then compresses documents to only the query-relevant parts, reducing context noise.
Similarity cutoff	<pre>post = SimilarityPostprocessor(similarity_cutoff=0.8)</pre>	Drops nodes below a minimum similarity threshold before synthesis.
Deduplication	<pre>unique = list({d.page_content: d for d in docs}.values())</pre>	Removes duplicate chunks before feeding context to the LLM.
TopN	<pre>top_n=10</pre>	Limits reranker output to the highest-scoring items only.

TABLE 9

Query Engines and Answer Grounding

This is the generation half of RAG — the components that take retrieved context and turn it into a written answer. Grounding is the key idea: instructing the model to answer only from the provided evidence is the core defense against hallucination, and the citation engines tie each claim back to its source so readers can verify it.

Engine	Example	Description
RetrieverQueryEngine	<pre>qe = RetrieverQueryEngine.from_args(retriever=retriever) resp = qe.query("...")</pre>	LlamaIndex query engine that retrieves nodes then synthesizes a response .
CitationQueryEngine	<pre>qe = CitationQueryEngine.from_args(index=index) resp = qe.query("...")</pre>	Generates answers with inline source citations anchored to retrieved chunks.
Grounding	"Answer only using the provided context."	- Forces the LLM to base claims on retrieved evidence rather than latent knowledge - core anti-hallucination mechanism.
PromptTemplate	<pre>prompt = ChatPromptTemplate.from_messages([("system", "..."), ("human", "{question}")])</pre>	Parameterizes prompts so retrieval context and user input can be inserted reliably.
ResponseMode	<pre>response_mode=ResponseMode.COMPACT</pre>	Controls how retrieved text is composed into prompts and how answers are formed.
ContextWindow	<pre>max_output_tokens=512</pre>	Limits generation length (and indirectly budgets room for retrieved context).
Streaming	<pre>stream=True</pre>	- Streams partial tokens while a completion is being generated - reduces perceived latency.
CiteGranularity	<pre>citation_chunk_size=512</pre>	Sets the chunk size used to form citation units for per-source attribution.

TABLE 10

Advanced RAG Architectures

Beyond the basic retrieve-then-generate loop sit the patterns that define modern production RAG. Most share one move: letting the system decide how to retrieve rather than hard-coding a single pass. Agentic RAG treats retrieval as a tool the LLM calls, corrective and self-RAG grade their own evidence, multi-hop chains several retrievals, and GraphRAG retrieves over a knowledge graph instead of flat chunks.

Pattern	Example	Description
Agentic RAG	<pre>tools=[search_knowledge_base]; agent.run(question)</pre>	• LLM decides when, what, and how to retrieve as a tool call • handles multi-step and conditional retrieval needs.
MCP-Based Retrieval	<pre>tools = mcp_client.list_tools(); agent.run(question, tools=tools)</pre>	• Exposes a knowledge base as a Model Context Protocol server so any MCP-compatible agent can call retrieval without bespoke integration code • the standard way production agentic RAG systems wire retrieval into a tool-calling loop.
GraphRAG	<pre>graphrag.query(query_type="local", query="...")</pre>	Microsoft's approach: extracts a knowledge graph from the corpus, builds community summaries, and retrieves via local, global, or DRIFT (community-informed local) search.
Corrective RAG (CRAG)	<pre>grade = evaluator.score(doc, q); if grade == "incorrect": web_search(q)</pre>	Lightweight evaluator grades each retrieved document (correct/ambiguous/incorrect) and triggers web search on failures.
Self-RAG	<pre># model uses reflection tokens: [Retrieve], [ISREL], [ISSUP]</pre>	Trains a single LM to adaptively retrieve on-demand and self-critique retrieved passages and its own generations.
Multi-Hop RAG	<pre>for hop in range(MAX_HOPS): docs=retrieve(q); q=refine(q,docs)</pre>	• Chains multiple retrieval steps where each hop's results inform the next query • needed for questions spanning multiple documents.
Contextual Retrieval	<pre>chunk = f"{llm_context(doc, chunk)}\n\n{chunk}"; embed(chunk)</pre>	• Anthropic technique: prepends chunk-specific context before embedding and BM25 indexing • reduces retrieval failures by 49-67% combined with reranking.
Adaptive RAG	<pre>route = classifier(query); pipeline = routes[route]</pre>	A query complexity classifier routes each query to the appropriate pipeline — no retrieval, single-hop, or multi-hop — saving cost on simple queries.

TABLE 11

Storage, Persistence, and Caching

The plumbing that keeps a RAG system durable and affordable lives here — where vectors, full documents, and index structures get persisted so you don't rebuild them on every restart. The caching and incremental-indexing rows matter most at scale: re-embedding only changed documents and caching repeat embeddings are where real cost savings come from.

Store	Example	Description
VectorStore	<pre>vectors = embed(texts); upsert(vectors, metadata)</pre>	Persists embeddings + payloads for similarity search at retrieval time.
DocStore	<pre>docstore.add_documents(docs)</pre>	• Stores full documents (separate from chunk/node indexes) • used by ParentDocumentRetriever.
IndexStore	<pre>index_store.persist(persist_dir=".storage")</pre>	Persists index metadata/structures for reload without rebuilding.
StorageContext	<pre>ctx = StorageContext.from_defaults(persist_dir=".storage")</pre>	Bundles storage backends used by an index/query pipeline.
Persistence	<pre>CREATE TABLE items(id bigserial, content text, embedding vector(1536));</pre>	Makes embeddings/queryable data durable in a database.
Incremental Indexing	<pre>if doc.hash != stored.hash: re_embed(doc)</pre>	Re-indexes only changed documents to keep the vector store current without full rebuilds.

Store	Example	Description
EmbeddingCache	<code>cache_key = sha256(text)</code>	- Avoids recomputing embeddings for identical inputs - critical for cost control at scale.
Namespace	<code>index.query(namespace="prod", vector=q, top_k=10)</code>	Separates tenant or environment data within one vector index for multi-tenant isolation.
TTL	<code>SET key value EX 3600</code>	Expires cached retrieval/generation artifacts after a time-to-live window.

TABLE 12

RAG Evaluation Metrics and Frameworks

You can't improve what you can't measure, and RAG quality splits cleanly into two questions — did retrieval surface the right context, and did generation stay faithful to it. These metrics (faithfulness, context precision/recall, answer relevancy) pin down each side, and the frameworks below let you run them in CI against a golden dataset so a pipeline change can be graded before it ships.

Metric	Example	Description
Faithfulness	<code>faithfulness ∈ [0,1]</code>	- Measures consistency with retrieved context - the primary anti-hallucination metric.
Answer Relevancy	<code>answer_relevancy ∈ [0,1]</code>	Measures how well the answer addresses the question .
Context Precision	<code>context_precision ∈ [0,1]</code>	Measures whether retrieved contexts are useful for answering the question.
Context Recall	<code>context_recall ∈ [0,1]</code>	Measures how much of the needed information is present in retrieved contexts.
Factual Correctness	<code>factual_correctness ∈ [0,1]</code>	Measures whether the answer is factually correct against a reference.
Noise Sensitivity	<code>noise_sensitivity ∈ [0,1]</code>	Measures robustness to irrelevant context — does the answer degrade when noisy chunks are included?
Context Entities Recall	<code>context_entities_recall ∈ [0,1]</code>	Measures entity recall over retrieved context vs reference.
Aspect Critic	<code>aspect_critic</code>	LLM-judge style metric for assessing a specific aspect of the output (e.g., conciseness, harmlessness).
DeepEval	<code>@assert_test; def test_rag(): assert_llm(actual_output, expected_output, [AnswerRelevancyMetric()])</code>	pytest-compatible LLM testing framework with 50+ metrics across RAG, AI agents, and multi-turn chats - the most-downloaded framework in this category, designed for CI/CD quality gates.
Arize Phoenix	<code>px.launch_app(); tracer = register(project_name="rag")</code>	- Open-source AI observability platform - OpenTelemetry-based tracing with built-in RAG evaluators - self-hostable.
LangSmith	<code>client = langsmith.Client(); client.r un_on_dataset(dataset_name="rag-eval" , llm_or_chain=chain)</code>	- LangChain-native tracing and evaluation platform - deep visibility into chain execution steps and LLM calls.
Golden Dataset	<code>[{"question": q, "ground_truth": a, "source_docs": []}]</code>	- Curated Q&A set with approved answers and source documents - used to benchmark pipeline changes before deployment.

TABLE 13

Observability and Security

Once RAG is live you need to see inside it and defend it. The observability rows cover the OpenTelemetry GenAI spans and tracing tools that expose retrieval counts, token usage, and latency for debugging. The security rows map the OWASP LLM risks that hit RAG specifically: prompt injection riding in through retrieved context, the dedicated vector-and-embedding-weakness category covering cross-tenant leakage and embedding inversion, and corpus poisoning.

Signal	Example	Description
GenAI spans	<code>gen_ai.operation.name = "chat"</code>	Standardizes tracing semantics for GenAI operations (inference, retrieval, tools) via OpenTelemetry.
Retrieval span	<code>gen_ai.retrieval.count = 10</code>	Captures retrieval metadata (chunk count, latency, scores) to debug relevance vs latency tradeoffs.
Token metrics	<code>gen_ai.client.token.usage</code>	Records token usage to monitor cost and performance over time.
Langfuse	<code>langfuse = Langfuse(); trace = langfuse.trace(name="rag-query")</code>	Open-source LLM observability platform with trace-based debugging, evals, and a prompt management UI.
Prompt Injection	<code>"Ignore previous instructions and ..."</code>	Attacker-controlled input attempts to override system/developer intent via the retrieval context.
Vector and Embedding Weaknesses	<code>"resume with hidden white-on-white text: ignore prior instructions..."</code>	- OWASP's RAG-specific risk category: unauthorized cross-tenant retrieval, embedding-inversion attacks that recover source text, and hidden instructions smuggled inside indexed documents - mitigated with permission-aware vector stores and input validation before indexing.
Sensitive Information Disclosure	<code>"Print your hidden prompt"</code>	Leakage of secrets, system prompts, or private data via model outputs.
Data and Model Poisoning	<code>"Upload malicious docs into the KB"</code>	Corrupts the retrieval corpus or training data so the model is grounded in incorrect or malicious context.
Improper Output Handling	<code>render_html(llm_output)</code>	Treating model output as trusted without validation or sanitization can lead to downstream injection or execution.
Unbounded Consumption	<code>"Summarize 10MB of text"</code>	- Uncontrolled inference volume or oversized inputs drive excessive compute/cost, denial of service, or model/IP theft - mitigated with rate limits, input-size caps, and timeouts.

TABLE 14

Multimodal RAG

A huge share of real knowledge lives in PDFs full of tables, charts, and scanned layouts that text extraction mangles. Multimodal RAG sidesteps that by treating document pages as images — ColPali-style retrievers embed page images directly and a vision-language model reads them, increasingly replacing the old OCR-then-embed pipeline for complex documents.

Technique	Example	Description
ColPali	<code>model = RAGMultiModalModel.from_pretrained("vidore/colpali-v1.2"); model.index(pdf_folder)</code>	- VLM-based retriever that produces multi-vector embeddings directly from document page images via late interaction - no OCR needed.
ColQwen2	<code>model = ColQwen2.from_pretrained("vidore/colqwen2-v1.0"); scores = processor.score_multi_vector(q_emb, img_emb)</code>	- ColPali-style retriever built on Qwen2-VL-2B instead of PaliGemma, taking dynamic image resolutions rather than a fixed resize - the backbone most current multimodal-RAG tooling ships by default.

Technique	Example	Description
Byaldi	<pre>docs_model = RAGMultiModalModel.from_pretrained("vidore/colpali-v1.2"); results = docs_model.search(query, k=3)</pre>	Python library wrapping ColPali-family models with a familiar API for indexing PDFs and searching by visual content.
VLM Synthesis	<pre>vl_model.generate(images=retrieved_pages, text=query)</pre>	Uses a Vision Language Model (e.g., Qwen2-VL, GPT-4V) to answer based on retrieved document page images.
OCR Pipeline	<pre>text = ocr_engine.extract(page_image); embed(text)</pre>	- Traditional text-extraction pipeline before embedding superseded by ColPali-style retrievers for documents with complex layouts, tables, and figures.

References

Official Documentation

- https://docs.llamaindex.ai/en/stable/module_guides/loading/documents_and_nodes/
- https://docs.llamaindex.ai/en/stable/module_guides/deploying/query_engine/
- https://docs.llamaindex.ai/en/stable/examples/query_engine/citation_query_engine/
- https://docs.llamaindex.ai/en/stable/api_reference/node_parsers/sentence_splitter/
- https://docs.llamaindex.ai/en/stable/api_reference/postprocessor/similarity/
- https://docs.llamaindex.ai/en/stable/module_guides/storing/docstores/
- https://docs.llamaindex.ai/en/stable/module_guides/storing/storage_context/
- https://docs.llamaindex.ai/en/stable/examples/query_engine/sub_question_query_engine/
- https://python.langchain.com/api_reference/core/retrievers/langchain_core_retrievers.BaseRetriever.html
- https://python.langchain.com/api_reference/text_splitters/character/langchain_text_splitters.character.CharacterTextSplitter.html
- https://python.langchain.com/api_reference/text_splitters/markdown/langchain_text_splitters.markdown.MarkdownHeaderTextSplitter.html
- https://python.langchain.com/docs/how_to/semantic-chunker/
- https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.contextual_compression.ContextualCompressionRetriever.html
- https://python.langchain.com/api_reference/langchain/retrievers/langchain_retrievers.parent_document_retriever.ParentDocumentRetriever.html
- https://python.langchain.com/docs/how_to/routing/
- https://python.langchain.com/docs/concepts/text_splitters/
- <https://platform.openai.com/docs/guides/prompt-engineering>
- <https://platform.openai.com/docs/guides/text-generation>
- <https://docs.cohere.com/reference/rerank>
- <https://docs.cohere.com/reference/embed>
- <https://docs.voyageai.com/docs/flexible-dimensions-and-quantization>
- <https://faiss.ai/index.html>
- https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexFlatL2.html
- https://faiss.ai/cpp_api/namespace/namespacefaiss.html
- <https://docs.pinecone.io/>
- <https://qdrant.tech/documentation/>
- <https://qdrant.tech/articles/binary-quantization-openai/>
- <https://docs.weaviate.io/weaviate/search/filters>
- <https://docs.weaviate.io/>
- <https://docs.trychroma.com/>
- <https://www.elastic.co/docs/reference/elasticsearch/index-settings/similarity>
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_entities_recall/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/answer_relevancy/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/noise_sensitivity/
- https://python.langchain.com/docs/concepts/prompt_templates/
- https://python.langchain.com/docs/how_to/vectorstore_retriever/
- <https://platform.openai.com/docs/guides/embeddings>
- <https://platform.openai.com/docs/guides/streaming-responses>
- <https://docs.cohere.com/docs/rerank-overview>
- <https://docs.voyageai.com/docs/embeddings>
- <https://ai.google.dev/gemini-api/docs/embeddings>
- https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexFlatIP.html
- https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexPQ.html
- <https://github.com/pgvector/pgvector>
- <https://docs.pinecone.io/guides/indexes/use-namespaces>
- <https://qdrant.tech/documentation/guides/quantization/>
- <https://docs.weaviate.io/weaviate/search/hybrid>
- <https://docs.weaviate.io/weaviate/concepts/data>
- <https://milvus.io/docs/overview.md>
- <https://lancedb.github.io/lancedb/>
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/answer_relevancy/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/
- https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/noise_sensitivity/

63. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/aspect_critic/
65. <https://docs.confident-ai.com/>
67. <https://docs.smith.langchain.com/>
69. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
71. <https://opentelemetry.io/docs/specs/semconv/registry/attributes/gen-ai/>
73. <https://genai.owasp.org/llm-top-10/>
75. <https://genai.owasp.org/llmrisk/llm02-insecure-output-handling/>
77. <https://genai.owasp.org/llmrisk/llm05-supply-chain-vulnerabilities/>
79. <https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/>
81. <https://microsoft.github.io/graphrag/>
83. <https://www.microsoft.com/en-us/research/project/graphrag/>
85. <https://huggingface.co/BAAI/bge-m3>
87. <https://huggingface.co/intfloat/e5-mistral-7b-instruct>
89. <https://huggingface.co/vidore/colqwen2-v1.0>
91. <https://github.com/AnswerDotAI/byaldi>
93. <https://learn.microsoft.com/en-us/azure/developer/ai/advanced-retrieval-augmented-generation>
95. <https://cohere.com/blog/rerank-4>
64. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/factual_correctness/
66. <https://phoenix.arize.com/>
68. <https://langfuse.com/docs>
70. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-metrics/>
72. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
74. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
76. <https://genai.owasp.org/llmrisk/llm04-model-denial-of-service/>
78. <https://genai.owasp.org/llmrisk/llm08-excessive-agency/>
80. <https://redis.io/docs/latest/develop/data-types/strings/>
82. <https://microsoft.github.io/graphrag/query/overview/>
84. <https://modelcontextprotocol.io/introduction>
86. <https://huggingface.co/BAAI/bge-reranker-v2-m3>
88. https://huggingface.co/learn/cookbook/multimodal_rag_using_document_retrieval_and_vlms
90. <https://github.com/stanford-futuredata/ColBERT>
92. <https://supabase.com/docs/guides/database/extensions/pgvector>
94. <https://www.anthropic.com/news/contextual-retrieval>

Academic Papers

96. <https://arxiv.org/abs/2005.11401>
98. <https://arxiv.org/abs/2212.10496>
100. <https://arxiv.org/abs/2310.11511>
102. <https://arxiv.org/abs/2501.09136>
104. <https://arxiv.org/abs/2403.14403>
106. <https://arxiv.org/abs/1603.09320>
108. <https://arxiv.org/pdf/2409.04701>
110. <https://dl.acm.org/doi/10.1145/1571941.1572114>
112. <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>
114. <https://openreview.net/forum?id=ogjBpZ8uSi>
97. <https://arxiv.org/pdf/2005.11401>
99. <https://arxiv.org/pdf/2212.10496>
101. <https://arxiv.org/abs/2401.15884>
103. <https://arxiv.org/abs/2407.01449>
105. <https://arxiv.org/abs/2310.06117>
107. <https://arxiv.org/abs/2409.04701>
109. <https://arxiv.org/abs/2511.00444>
111. <https://dl.acm.org/doi/10.1145/290941.291025>
113. <https://aclanthology.org/2023.acl-long.99/>
115. <https://openreview.net/forum?id=74QmBTV0Zf>

Technical Blogs & Tutorials

116. <https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables>
118. <https://weaviate.io/blog/hybrid-search-explained>
120. <https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/>
122. <https://qdrant.tech/blog/rag-evaluation-guide/>
124. <https://www.mongodb.com/docs/atlas/ai-integrations/langchain/parent-document-retrieval/>
126. <https://www.sbert.net/examples/applications/cross-encoder/README.html>
128. <https://blog.prem.ai/blog/rag-chunking-strategies-the-2026-benchmark-guide/>
130. <https://blog.prem.ai/blog/rag-evaluation-metrics-frameworks-testing-2026/>
132. <https://www.firecrawl.dev/blog/best-chunking-strategies-rag>
134. <https://callsphere.tech/blog/rag-architecture-patterns-2026-retrieval-augmented-generation>
136. https://dev.to/young_gao/rag-is-not-dead-advanced-retrieval-patterns-that-actually-work-in-2026-2gbo
138. <https://procogia.com/unlocking-rags-potential-mastering-advanced-techniques-part-1/>
140. <https://www.meilisearch.com/blog/graph-rag>
142. <https://weaviate.io/blog/late-interaction-overview>
144. <https://deepeval.com/blog/deepeval-vs-trulens>
146. <https://www.getmaxim.ai/articles/the-5-best-rag-evaluation-tools-you-should-know-in-2026/>
117. <https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/>
119. <https://www.postgresql.org/about/news/pgvector-080-released-2952/>
121. <https://redis.io/blog/get-better-rag-responses-with-ragas/>
123. <https://www.datadoghq.com/blog/llm-otel-semantic-convention/>
125. <https://www.sbert.net/docs/cross-encoder.html>
127. <https://nlp.stanford.edu/IR-book/html/htmledition/query-expansion-1.html>
129. <https://www.prem.ai/blog/rag-chunking-strategies-the-2026-benchmark-guide/>
131. <https://www.prem.ai/blog/best-embedding-models-for-rag-2026-ranked-by-mteb-score-cost-and-self-hosting/>
133. <https://www.firecrawl.dev/blog/best-vector-databases>
135. <https://rapidclaw.dev/blog/rag-architecture-ai-agents-guide-2026>
137. <https://encore.dev/articles/best-vector-databases>
139. <https://www.lancedb.com/blog/modified-rag-parent-document-bigger-chunk-retriever-62b3d1e79bc6>
141. <https://graphrag.com/concepts/intro-to-graphrag/>
143. <https://blog.gopenai.com/the-fidelity-crisis-in-rag-why-late-interaction-colbert-is-the-4k-image-of-search-vs-e978d96b25b8>
145. <https://deepeval.com/blog/top-5-llm-evaluation-frameworks>
147. <https://rhesis.ai/post/best-llm-evaluation-testing-tools>

- 148. <https://ragflow.io/blog/rag-review-2025-from-rag-to-context>
- 150. <https://www.turingpost.com/p/ragtypes>
- 152. <https://huggingface.co/blog/manu/colpali>
- 154. <https://www.dev.to/polliog/postgresql-as-a-vector-database-when-to-use-pgvector-vs-pinecone-vs-weaviate-4kfi>

GitHub Repositories & Code Examples

- 156. <https://github.com/langchain-ai/langchain>
- 158. <https://github.com/facebookresearch/faiss>
- 160. <https://github.com/microsoft/graphrag>
- 162. <https://github.com/stanford-futuredata/ColBERT>
- 164. <https://github.com/jina-ai/late-chunking>

Video Resources

- 166. <https://www.youtube.com/watch?v=j66Db1SB1YY>
- 168. <https://www.youtube.com/watch?v=VfjIYjYFVt4>
- 170. <https://www.youtube.com/watch?v=-1zMU1a625E>
- 172. <https://www.youtube.com/watch?v=buzWGXOydD8>

Industry Best Practice Guides & Books

- 173. <https://www.aws.amazon.com/blogs/machine-learning/contextual-retrieval-in-anthropic-using-amazon-bedrock-knowledge-bases/>
- 174. <https://nemorize.com/roadmaps/2026-modern-ai-search-rag-roadmap/lessons/agent-ric-rag-systems>
- 175. <https://pecollective.com/tools/best-embedding-models/>
- 176. <https://pecollective.com/tools/text-embedding-models-compared/>

- 149. <https://blog.starmorph.com/blog/rag-techniques-compared-best-practices-guide>
- 151. <https://jina.ai/news/late-chunking-in-long-context-embedding-models/>
- 153. <https://milvus.io/blog/choose-embedding-model-rag-2026.md>
- 155. <https://ackuity.ai/blog/mitigating-vector-and-embedding-weaknesses>

- 157. https://github.com/run-llama/llama_index
- 159. <https://github.com/pgvector/pgvector>
- 161. <https://github.com/AnswerDotAI/byaldi>
- 163. <https://github.com/PranavGovindu/Self-Corrective-Agent-ric-RAG>
- 165. <https://github.com/microsoft/multi-modal-rag-with-colpali>

- 167. https://www.youtube.com/watch?v=nMCII_xtUbw
- 169. <https://www.youtube.com/watch?v=0fackgiKTiA>
- 171. <https://www.youtube.com/watch?v=vT-DpLv29Q>